MICROPROCESADORES · UPB · RETO DE LA UNIDAD 3

Driver SPI bare-metal para la STM32F407

Lectura del acelerómetro ADXL345 por SPI2, escrita a nivel de registros, con los datos en pantalla por SWO

Gildardo Estevan Restrepo

Marco Aurelio Guardia

Semestre 2026-2

EL RETO

Un driver propio, de los registros a la aplicación

Driver bare-metal

SPI2 configurado directamente en sus registros: sin STM32Cube HAL, sin LL y sin CMSIS.

Prueba de concepto

Lectura continua del acelerómetro ADXL345: aceleración en X, Y y Z, en miligravedades.

Visualización por SWO

printf sale por el pin PB3 hacia el depurador J-Link, sin usar ninguna USART.

Criterio que seguimos: poder explicar cada bit que escribimos en un registro.

PROTOCOLO SPI

Un bus síncrono, full-duplex y de cuatro líneas

Síncrono

El maestro genera el reloj SCK. No hay baud rate que acordar, como en UART.

Full-duplex

En cada pulso de reloj sale un bit por MOSI y entra otro por MISO, al mismo tiempo.

Maestro–esclavo

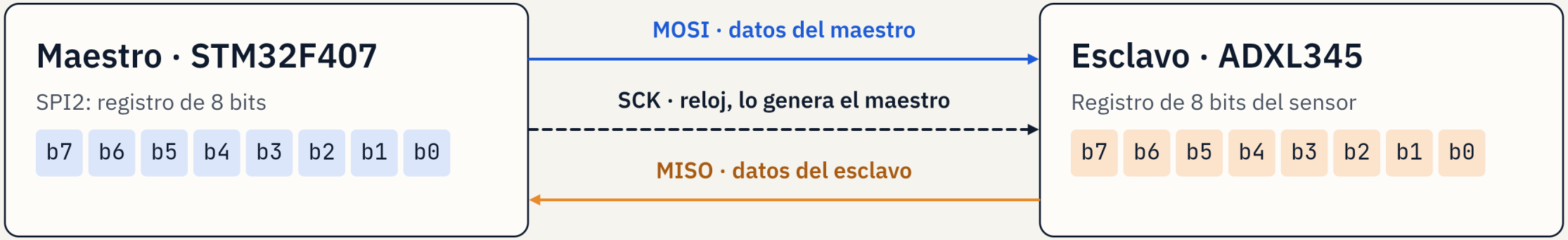
Solo el maestro inicia. El esclavo responde únicamente mientras su CS está en bajo.

Línea	Sentido	Función
SCK	Maestro → esclavo	Reloj de la transferencia
MOSI	Maestro → esclavo	Datos que envía el maestro
MISO	Esclavo → maestro	Datos que responde el esclavo
CS	Maestro → esclavo	Selección, activa en bajo

Con CS en alto, el esclavo ignora el reloj y suelta MISO. Por eso varios esclavos pueden compartir SCK, MOSI y MISO, cada uno con su propio CS.

PROTOCOLO SPI

Dos registros de desplazamiento en anillo



Tras 8 pulsos de reloj, los dos registros intercambiaron su contenido.

Leer implica escribir

Para recibir, el maestro envía un byte dummy (0x00) que genera los 8 pulsos de reloj.

Escribir implica recibir

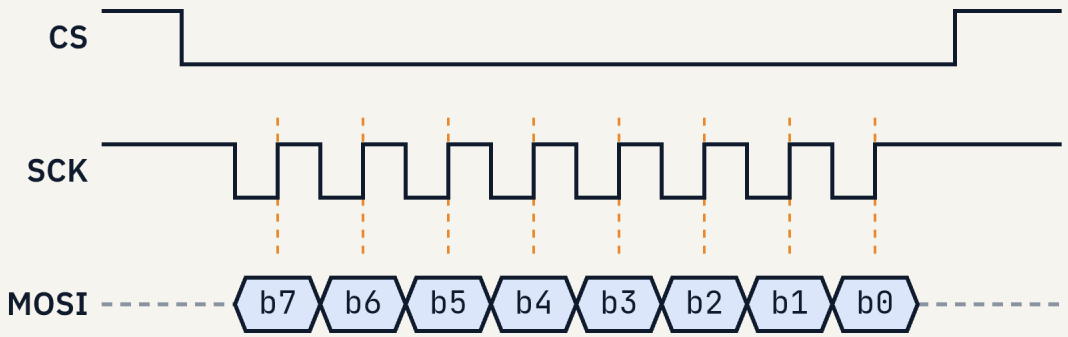
El byte que llega se lee siempre, aunque no sirva, para no provocar overrun (OVR).

PROTOCOLO SPI

CPOL y CPHA: los cuatro modos de reloj

Modo	CPOL	CPHA	Reposo	Captura
0	0	0	Bajo	Subida
1	0	1	Bajo	Bajada
2	1	0	Alto	Bajada
3	1	1	Alto	Subida

Maestro y esclavo deben usar el mismo modo; si no, los bits se leen corridos. La fila resaltada es el **modo 3**, el que exige el ADXL345.



Modo 3: el reloj reposa en alto, el dato cambia en el flanco de bajada y se captura en el de subida (líneas naranjas).

HARDWARE

El periférico SPI2 de la STM32F407

Dato	Valor
Periférico	SPI2, dirección base 0x4000 3800
Bus	APB1; su reloj se enciende en RCC_APB1ENR, bit 14
Reloj del bus	16 MHz: oscilador HSI, sin PLL
Divisor	BR = 011, es decir ÷16
Reloj SCK	1 MHz (el ADXL345 admite hasta 5 MHz)
Trama	8 bits, MSB primero, full-duplex
Atención	Polling: CR2 = 0, sin interrupciones ni DMA

VELOCIDAD DEL RELOJ

```
fSCK = fPCLK1 / 2^(BR+1)
= 16 MHz / 16 = 1 MHz
```

SPI_DR es una sola dirección con dos buffers: escribirla llena el de transmisión y leerla vacía el de recepción.

HARDWARE

Conexiones de la prueba de concepto

GY-291	STM32F407	Función
VCC	3V3	Alimentación
GND	GND	Tierra
CS	PB12	Chip select (GPIO de salida)
SCL	PB13	SPI2_SCK (AF5)
SDO	PB14	SPI2_MISO (AF5)
SDA	PB15	SPI2_MOSI (AF5)

Depurador J-Link por SWD: PA13 (SWDIO), PA14 (SWCLK) y SWO en PB3.

Por qué SPI2 y no SPI1

- ⚠ SPI1 en PA5–PA7 choca con los LEDs de la placa en PA6 y PA7, que cargarían MISO.
- ⚠ SPI1 en PB3–PB5 comparte líneas con la flash W25Q16 y el zócalo NRF24L01.
- ✓ Con SPI2, PB3 queda libre para la salida SWO del depurador.
- ✓ El CS va en PB12 como GPIO normal, manejado por software.

ARQUITECTURA DE SOFTWARE

Cuatro capas: cada una llama solo a la de abajo

main.c	Aplicación: lee el sensor, imprime y reintenta si falla
adxl345.c/.h	Driver del sensor: sus registros y la conversión a mg
drv_spi.c/.h	API SPI: arma tramas completas, de bajar CS a subirlo
ll_spi.c/.h	Bajo nivel: la única capa del SPI que toca registros (RCC, GPIOB, SPI2)
swo.c/.h	Módulo aparte: salida de printf por SWO

EJEMPLO: UNA LECTURA DE LOS EJES



DOMINIO DE REGISTROS

SPI_CR1 bit a bit

Bit	Nombre	Valor	Para qué
11	DFF	0	Trama de 8 bits
9	SSM	1	NSS manejado por software
8	SSI	1	NSS interno en alto: evita MODF
7	LSBFIRST	0	MSB primero
6	SPE	1	Habilita el SPI; se escribe al final
5:3	BR	011	fPCLK1 / 16 = 1 MHz
2	MSTR	1	Maestro: genera SCK
1	CPOL	1	Reloj en reposo alto
0	CPHA	1	Captura en el 2.º flanco: modo 3

VALOR FINAL

0x035F

`0000 0011 0101 1111`

Los bits de configuración se escriben con SPE = 0 y SPE se pone en 1 al final (RM0090, §28.3.3).

DOMINIO DE REGISTROS

Secuencia de inicialización: LL_SPI_Init()

1. **Encender relojes**
   RCC_AHB1ENR.GPIOBEN y RCC_APB1ENR.SPI2EN, más una lectura de relleno: el reloj tarda unos ciclos en quedar activo (errata ES0182).

2. **Reset de SPI2**
   RCC_APB1RSTR.SPI2RST: cada inicialización parte de los valores de fábrica, y sirve para recuperar el bus después de un error.

3. **Pines de GPIOB**
   CS = 1 antes de volverlo salida (sin selección falsa); PB13–PB15 en AF5; pull-up en SCK y MISO. Campo por campo, para no tocar PB3 y PB4 del depurador.

4. **Configurar SPI_CR1 con SPE = 0**
   SSM, SSI, MSTR, CPOL, CPHA y BR = 011: SPI_CR1 = 0x031F.

5. **Habilitar**
   SPE = 1, SPI_CR1 = 0x035F. El bus queda listo, con CS en alto.

DOMINIO DE REGISTROS

Transferir un byte: LL_SPI_TransferByte()

1 · Esperar
`TXE = 1`

→

2 · Escribir
`SPI_DR = tx`

→

3 · Esperar
`RXNE = 1`

→

4 · Leer
`rx = SPI_DR`

Bit	SPI_SR	Significado
7	BSY	Transfiriendo: esperar 0 antes de subir CS
6	OVR	Llegó un byte sin haber leído el anterior
1	TXE	Buffer de transmisión vacío
0	RXNE	Hay un byte recibido para leer

Antes de subir CS

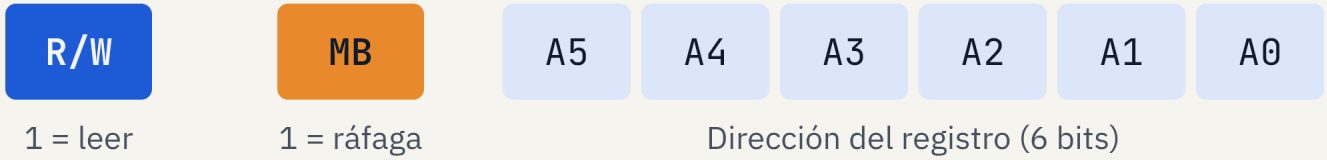
Esperar BSY = 0; si no, se corta el último byte a la mitad (RM0090, §28.3.8).

Ninguna espera es infinita

Cada bandera tiene un límite de 10 000 vueltas. Si vence, la función devuelve 0xFF y guarda LL_SPI_ERR_TIMEOUT.

PROTOCOLO DEL SENSOR

Tramas del ADXL345: un byte de comando y los datos



0xF1

leer + ráfaga + 0x31

Operación	MOSI	Respuesta útil (MISO)
Leer DEVID (0x00)	80 00	2.º byte = E5: el sensor está
Formato ±16 g, resolución completa	31 0B	Ninguna
Iniciar la medición (POWER_CTL)	2D 08	Ninguna
Leer formato y ejes, desde 0x31	F1 + 7 × 00	0B, X0 X1, Y0 Y1, Z0 Z1

Cada eje: 16 bits en complemento a 2, primero el byte bajo. mg = raw × 1000 / 256, es decir 3,9 mg por LSB.

ROBUSTEZ

SPI no tiene ACK: el maestro lee aunque no haya nadie del otro lado.

SENSOR DESCONECTADO

MISO queda en alto por el pull-up y DATA_FORMAT se lee como 0xFF.

`ERR_CONFIG`

SENSOR REINICIADO

Vuelve en reposo y DATA_FORMAT se lee como 0x00, no como 0x0B.

`ERR_CONFIG`

BUS BLOQUEADO

Una bandera no cambia a tiempo y vence la espera de ll_spi.

`ERR_SPI`

En cualquier error, main reinicia el SPI y el sensor en la vuelta siguiente: el sistema se recupera solo, sin reiniciar la placa.

VISUALIZACIÓN

printf sin USART: los datos salen por SWO



Registro	Valor	Para qué
DEMCR.TRCENA	1	Enciende ITM y TPIU
TPIU_SPPR	2	Salida serie NRZ
TPIU_ACPR	7	16 MHz / 8 = 2 MHz
DBGMCU_CR.TRACE_IOEN	1	Asigna PB3 a la traza
ITM_LAR	0xC5ACCE55	Abre el candado del ITM
ITM_TCR, ITM_TER	ITMENA, 1	Habilita el ITM y el puerto 0

En el depurador

- Core Clock = 16 MHz
- SWO Clock = 2000 kHz
- Puerto 0 del ITM y Start Trace

Si no coinciden con el firmware, la consola muestra caracteres extraños. Basado en la implementación de SWO del profesor, adaptada al proyecto.

DEMOSTRACIÓN EN VIVO

Cuatro pruebas en la placa

1 Arranque

Iniciamos la depuración y abrimos la consola SWV.

Esperado: «ADXL345 listo» y una línea X, Y, Z por lectura.

2 Los tres ejes

Giramos el sensor sobre cada cara.

Esperado: el eje vertical cerca de ±1000 mg; los otros, cerca de 0.

3 Desconexión

Soltamos el cable de SDO con el programa corriendo.

Esperado: ERR_CONFIG y el contador de errores sube; nada se bloquea.

4 Recuperación

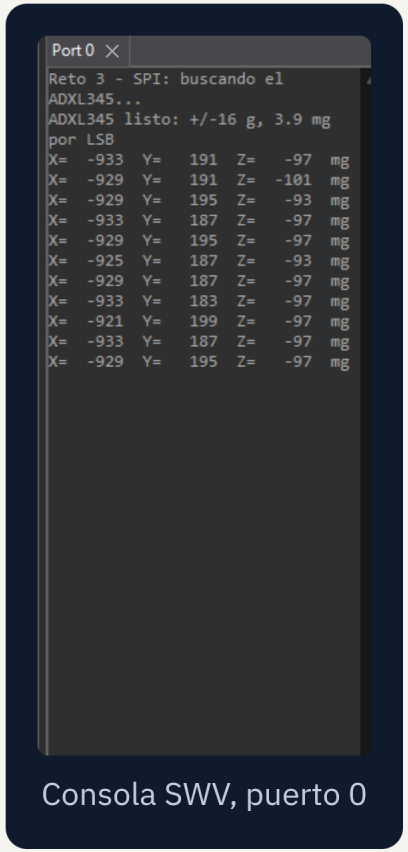
Volvemos a conectar el cable.

Esperado: las lecturas regresan solas, sin reiniciar la placa.

Plan B: si la consola SWV falla, las mismas variables se ven en Live Expressions: accel, lecturas, errores y estado.

RESULTADOS EN LA PLACA

Lecturas reales del ADXL345



Consola SWV, puerto 0

≈ 957 mg

Magnitud de (−933, 191, −97) mg. Es la gravedad, 1 g, con el error de fábrica del sensor.

3,9 mg

Las lecturas cambian de a un paso: es la resolución de 1 LSB a ±16 g.

X ≈ −933

Casi toda la gravedad cae en X: el módulo estaba de canto en la protoboard.

Conclusión: lo que llega por SPI es aceleración real, no basura del bus.

REPOSITORIO

GitHub: una rama por etapa y revisión cruzada

ESTRUCTURA

```
/src código fuente (.c)
/inc cabeceras (.h)
/_docs enunciado, teoría, datasheets
README API, conexiones, SWO
```

github.com/GildardoRestrepo/Reto_3_SPI

PR	Etapa	Autor	Fusionó
#1	Bajo nivel: ll_spi	Marco	Gildardo
#2	Alto nivel: drv_spi y adxl345	Gildardo	Marco
#3	Detección de desconexión	Gildardo	Marco
#4	Visualización por SWO	Marco	Gildardo

Flujo: rama por etapa, Pull Request, revisión del otro integrante y merge a main. El PR #3 nació de un comentario de revisión en el #2.

CIERRE

Lo que nos llevamos

01 Cada bit de SPI_CR1 tiene una razón: configurar el SPI desde cero obliga a entender relojes, pines y banderas.

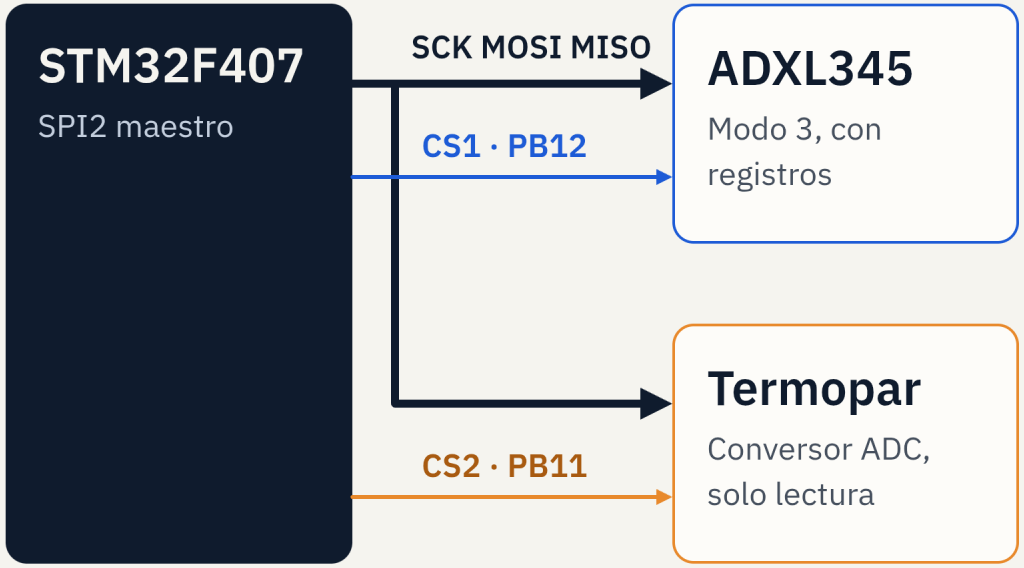
02 Separar el bajo nivel de la API nos dejó trabajar en paralelo, cada uno en su capa.

03 SPI no confirma nada: la robustez la pone el software, verificando lo que responde el sensor en cada lectura.

¿Preguntas?

TRABAJO FUTURO

Dos sensores en el mismo bus SPI



SCK, MOSI y MISO son compartidos; cada sensor tiene su propio CS y nunca hay dos CS en bajo a la vez.

UNA VUELTA DEL CICLO

1. **Leer el ADXL345**
   Modo 3, baja CS1, comando 0xF1 y 7 bytes, sube CS1.
2. **Leer el termopar**
   Cambia al modo del conversor con SPE = 0, baja CS2, lee 16 o 32 bits sin enviar comando, sube CS2.
3. **Mostrar**
   printf de X, Y, Z en mg y de la temperatura en °C. El termopar se lee cada ~0,25 s, su tiempo de conversión.

QUÉ SE AGREGA AL CÓDIGO

ll_spi: CS con parámetro y cambio de modo con SPE = 0
drv_spi: cada trama dice a qué sensor va y en qué modo
temp.c (nuevo): bits a °C y aviso de termopar abierto